A1 Documentation

Group....1
Master Description....1
Slave Description....2
Deadlock and starvation....3

Group

Nicolò Faedo	Mat:0001132760
Alessandro Gaggioli	Mat:0001120931
Mirco Paltrinieri	Mat:0001132476
Luca Sabatini	Mat:0001140642

In this assignment the aim was to create an interprocess communication between a, so called, master and slave processes to be triggered by a signal sent by the user to the master process.
Following the reception of the signal the master process needs to send a random integer number N (between 0 and 19) to a slave process and the latter needs to activate a number of threads to work concurrently that take the number received and add it randomly to a slot inside a buffer (which is initialized at zero).

Master Description

The master firstly creates a **named pipe** to establish connection with the slave process and then waits for a signal given by the user. This signal is the **SIGUSR1** sent by typing `kill -10 <child_pid>` on the shell, then the payload is sent to the slave through the named pipe that is created in the directory "/tmp/<pipe_name>". The master process receives signals and sends numbers to the slave until the variable called **"terminated"** becomes true. The variable "terminated" is triggered by the accumulator variable when it reaches 100. A symbolic number (-1) is sent to the slave to inform the slave that the accumulator has reached its limit (100) and so there is no need to keep receiving data from the pipe.

The file descriptor of the named pipe is declared as global variable so that also the parent code can reach it, this is to ensure that after the correct reception of a signal (in our case consider **SIGUSR1**) and the following creation of the pipe, if a wrong signal is received, the parent has the possibility to close the pipe and then terminate.

The master automatically starts the slave process:

```
if(system(SLAVE_PATH)) {
    perror("Error in running the slave\n") ;
    pipe_destroyer(fd) ;
    exit(1) ;
}
```

)

This prevents the master from encountering an error in the case receives a **SIGUSR1** but there is no slave that has opened the reading end of the pipe causing an error.

Slave Description

Master slave synchronization is not necessary because the OS takes care of it through FIFO : the reading done by the slave on the reading end of the pipe is *blocking*, so it only reads on the pipe once a new element is sent.
The synchronization done on the slave process is based on the following monitor:

```
typedef struct {
    pthread_mutex_t m ; //mutex
    pthread_cond_t writer_w ; //conditional variable to writers queue
    pthread_cond_t main_w ; //conditional variable for main - writers queue
    pthread_barrier_t wr_bar ;//barrier for writers synchronization
    int wr_count ; //number of finished writers
    int wr_waiting ; //number of waiting writers
    bool new_data_arrived ; //flag for new data arrived
}monitor_t ;
```

1. Monitor entities: 1 buffer, 5 threads: writers
2. Monitor API's: mt_Init(), mt_Destroyer()
3. Priorities: no priorities
4. Queues:
    - Writer_w : queue in which threads wait to receive new data from main
    - Main_w : queue in which the main waits for all the threads to update the buffer
5. States:
    - Number of finished writers: wr_count
    - Number of waiting writers: wr_waiting
    - Flag that notify a new data arrived: new_data_arrived
6. Detailed design of methods:
    All writers must edit the buffer before one can edit it again: wr_bar

When a writer modifies the buffer it stops at the barrier to wait for the other threads to reach it as well, here writers will lay until the last thread arrives and notifies the main to fetch the new number from the pipe that will trigger another cycle of update of the buffer.

```c
//init the monitor items
void mt_init() {
    pthread_mutex_init(&mt.m,NULL) ;
    pthread_cond_init(&mt.main_w,NULL) ;
    pthread_cond_init(&mt.writer_w,NULL) ;
    pthread_barrier_init(&mt.wr_bar,NULL,N_WRITERS) ;
    mt.wr_count = 0 ;
    mt.wr_waiting = 0 ;
    mt.new_data_arrived = false ;
}
```

```c
//destroy monitor items
void mt_destroyer() {
    pthread_mutex_destroy(&mt.m) ;
    pthread_cond_destroy(&mt.main_w) ;
    pthread_cond_destroy(&mt.writer_w) ;
    pthread_barrier_destroy(&mt.wr_bar) ;
}
```

```c
void *writer(void *arg) {

    while(!terminated) {

        int target=rand()%BUF_SIZE;

        //taking the lock
        pthread_mutex_lock(&mt.m) ;

        /*
         * while main doesn't read a new data,
         * update the number of waiting writers
         * if the current writer is the fifth, signal the main to read a new value
         *
         * wait for the signal from main (when main read a value)
         *
         * update (decrease) the number of waiting writers
         */
        while(!mt.new_data_arrived) {
            mt.wr_waiting++ ;

            if(mt.wr_waiting==5)
                pthread_cond_signal(&mt.main_w) ;

            pthread_cond_wait(&mt.writer_w,&mt.m) ;
            mt.wr_waiting-- ;
        }
```

Deadlock and starvation

Deadlock is averted through the implementation of the monitor structure, consisting of a mutex lock and a conditional variable.

However, there is a potential for starvation if the user fails to send any signal. In such a scenario, the program could enter a state of starvation where all threads are waiting for the main to provide them with a new number, but this signal may never arrive.